

Proiect - Proiectare Software

Sistem de Gestiune pentru o Agenție de Rezervare a Biletelor de Avion

Analiza Arhitecturală a Microserviciului **Notificări**

Apetrei Diana-Andreea

Grupa: 30231

Mai 2026

Cuprins

1	Introducere și Contextul Sistemului	2
1.1	Harta de Context (Context Map)	2
2	Arhitectura Internă a Microserviciului Notificări	2
2.1	Diagrama de Pachete (Arhitectura Hexagonală)	2
2.2	Diagrama de Clase Detaliată	3
3	Design-ul Bazei de Date (Persistența)	4
4	Modelarea Domeniului (Domain-Driven Design)	5
4.1	Aggregate Root — Clasa Notificare	5
4.2	Value Objects și Tipizare Puternică	5
5	Șabloane de Proiectare (Design Patterns) Utilizate	6
5.1	Șablonul Adaptor (Adapter / Wrapper)	6
5.2	Șablonul Singleton (Gestionat prin IoC)	6
6	Concluzii	6

1 Introducere și Contextul Sistemului

Proiectul curent implementează un sistem backend complex pentru o agenție de bilete de avion. Pentru a asigura scalabilitatea, mentenabilitatea și independența echipelor de dezvoltare, arhitectura globală a fost divizată în **Microservicii**, ghidate de limitele contextuale (*Bounded Contexts*) din Domain-Driven Design (DDD).

1.1 Harta de Context (Context Map)

Sistemul global este împărțit în șase contexte principale, fiecare acționând ca un microserviciu independent: *Zboruri*, *Bilete*, *Utilizatori*, *Notificări*, *Statistici* și *Export*.

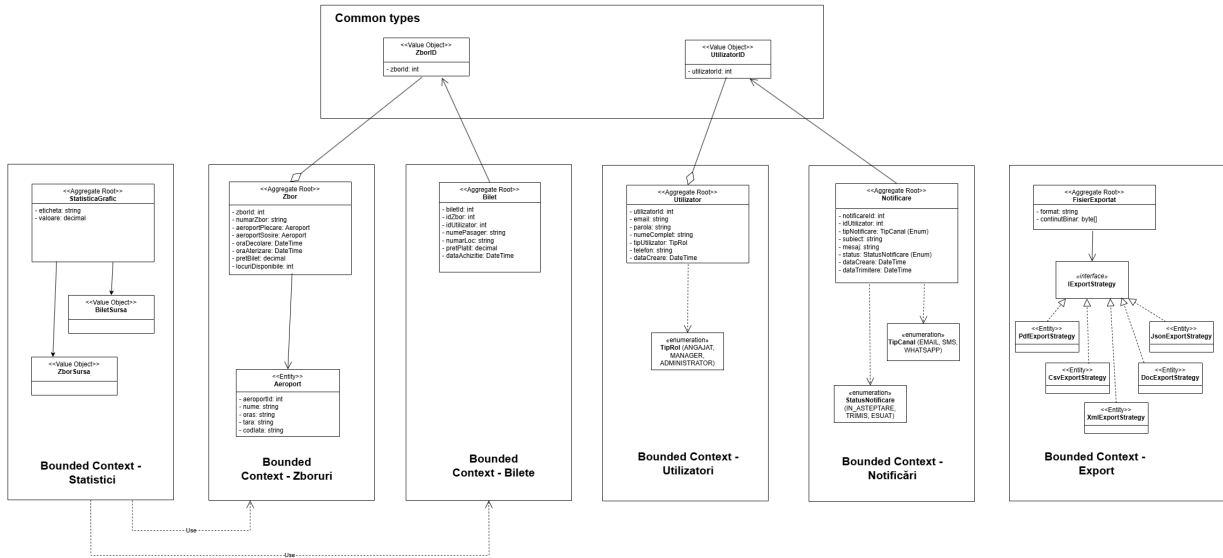


Figura 1: Harta de context a sistemului global de gestiune

Justificarea deciziei: Această decizie arhitecturală previne crearea unui monolit rigid. De exemplu, contextul **Notificări** (vizat exclusiv în acest document) este un domeniu de suport generic. El nu deține informații complexe despre rezervări sau pasageri, ci doar expune funcționalități de alertare (prin Email, SMS, WhatsApp). El primește evenimente de la microserviciul **Utilizatori** (de ex., modificarea securității contului) printr-un identificator neutru (`idUtilizator`) și se ocupă strict de persistența și distribuția mesajelor. Această decuplare permite modificarea furnizorilor de SMS/Email în viitor fără a afecta nucleul aplicației (Zboruri sau Bilete).

2 Arhitectura Internă a Microserviciului Notificări

2.1 Diagrama de Pachete (Arhitectura Hexagonală)

La nivel intern, microserviciul **Notificări** respectă cu strictețe principiul Inversiunii Dependințelor (*Dependency Inversion Principle*), specific Arhitecturii Hexagonale (sau Ports and Adapters).

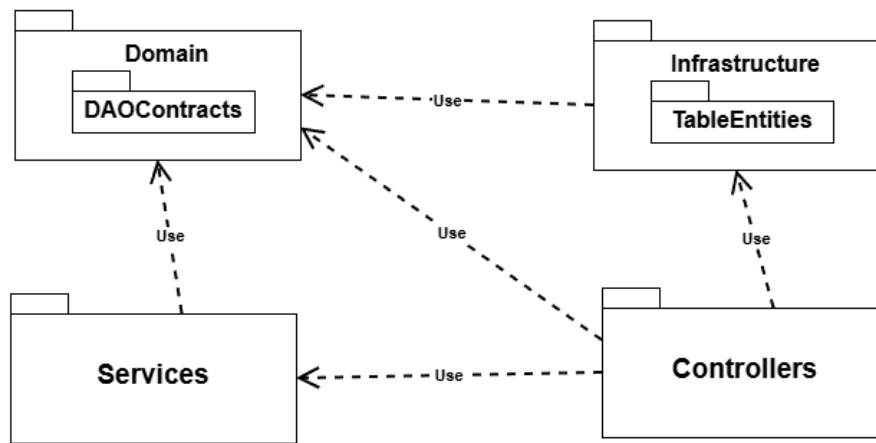


Figura 2: Diagrama de pachete: Organizarea internă pe straturi

Justificarea deciziei: Săgețile din figura 2 indică direcția dependențelor (-Use-). Se observă clar că inima aplicației este pachetul **Domain**, care conține doar cod Java pur (**Notificare**, **TipCanal**) și nu depinde de nimic. Pachetul **Infrastructure** (tehnologii volatile precum Spring Data JPA și **NotificareEntity**) depinde de **Domain** pentru a implementa contractul expus de acesta (**INotificareDAO**). Pachetul **Controllers** apelează **Services**, care orchestrează logica de notificare fără a ști detalii despre structura tabelor SQL.

2.2 Diagrama de Clase Detaliată

Pentru a expune în mod granular structura internă, tipurile de date, vizibilitatea atributelor și comportamentele metodelor, a fost mapată structura de clase a întregului modul.

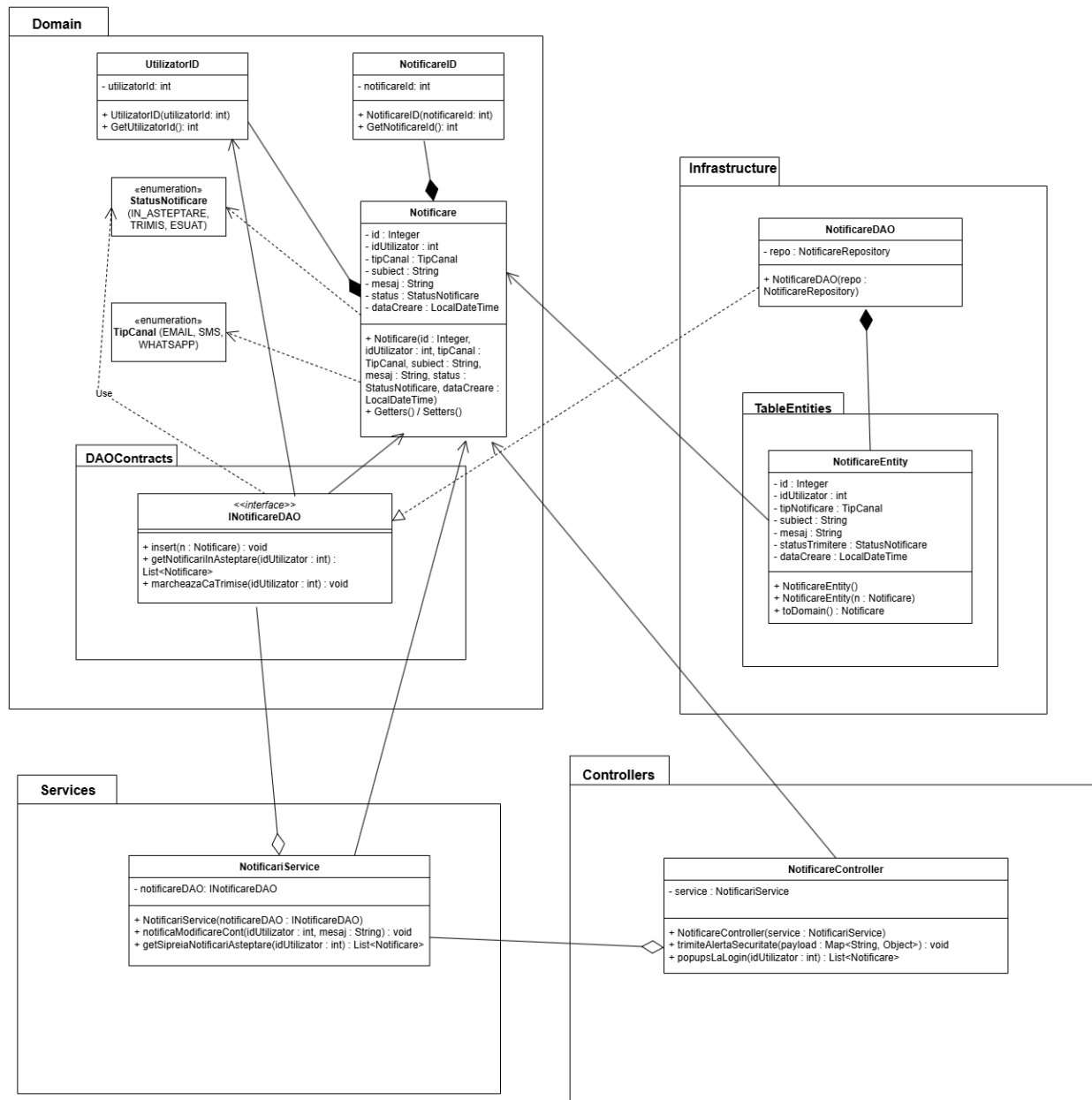


Figura 3: Diagrama de clase detaliată a microserviciului Notificări

Justificarea deciziei: Figura 3 oferă o vedere structurală completă. Ea evidențiază injectarea dependențelor (de la `NotificareController` către `NotificariService`, și mai departe către interfața `INotificareDAO`), precum și transformările (mapping-ul) realizate la nivelul infrastructurii între `Notificare` și `NotificareEntity`.

3 Design-ul Bazei de Date (Persistența)

Pentru salvarea datelor s-a optat pentru un model relațional (SQL), gestionat prin intermediul ORM-ului Hibernate.

notificare	
PK	<u>id: int</u>
	id_utilizator: int tip_notificare: ENUM('EMAIL', 'SMS', 'WHATSAPP') subiect: varchar(255) mesaj: varchar(255) status_trimitere: ENUM('TRIMIS', 'ESUAT', 'IN_ASTEPTARE') data_creare: datetime

Figura 4: Diagrama Entitate-Relație (ERD) pentru modulul Notificări

Justificarea deciziei: Figura 4 ilustrează o structură independentă de date. Tabela `notificare` conține coloana `id_utilizator` care acționează ca o *cheie străină logică*. Nu există o constrângere fizică de *Foreign Key* directă în baza de date, deoarece tabela utilizatorilor rezidă într-un alt microserviciu (o altă bază de date fizică). De asemenea, enum-urile (`tip_notificare`, `status_trimitere`) sunt persistate ca tip `VARCHAR` (`STRING`) pentru a preveni coruperea datelor în cazul schimbării ordinii constantelor în cod.

4 Modelarea Domeniului (Domain-Driven Design)

În interiorul nucleului **Domain**, logica de alertare este izolată prin tactici DDD.

4.1 Aggregate Root — Clasa Notificare

Clasa `Notificare` funcționează ca un *Aggregate Root*. Ea guvernează starea și validitatea oricărui mesaj creat:

- **Imuabilitatea structurală:** La instanțiere, un mesaj, un canal și un destinatar trebuie definite clar.
- **Ciclul de viață:** Constructorul garantează un *fallback* inteligent: dacă statusul nu este furnizat, se inițializează automat ca `IN_ASTEPTARE`, iar data curentă este înregistrată folosind `LocalDateTime.now()`.

4.2 Value Objects și Tipizare Puternică

Conceptul de canal (Email, SMS, WhatsApp) și stadiul trimiterii (În așteptare, Trimis, Eșuat) nu sunt reprezentate prin simple șiruri de caractere predispuse la greșeli (Typo-uri), ci prin concepte de domeniu **Enum** (`TipCanal`, `StatusNotificare`). ID-ul utilizatorului (`idUtilizator`) este modelat ca un **Value Object**, subliniind faptul că acest microserviciu nu deține conceptul de *Utilizator* (nu știe parola sau numele lui), ci doar îi cunoaște identitatea.

5 Șabloane de Proiectare (Design Patterns) Utilizate

5.1 Șablonul Adaptor (Adapter / Wrapper)

Pentru a realiza comunicarea între stratul de **Domain** și cel de **Infrastructure** (vezi Figura 2), am integrat pattern-ul Adaptor.

Explicație și Justificare: În cadrul arhitecturii noastre (Figura 3), interfața `INotificareDAO` reprezintă un **Port** definit de domeniul aplicației. Clasa `NotificareDAO` din infrastructură acționează ca un **Adaptor**. Rolul său este de a implementa acest contract, luând obiectele agnostice `Notificare` și convertindu-le (mapându-le) în `NotificareEntity`, pentru a putea folosi metodele magice din `NotificareRepository` (Spring Data JPA). Prin acest pattern, logica de business este complet protejată de detaliile bazei de date.

5.2 Șablonul Singleton (Gestionat prin IoC)

Implementarea șablonului Singleton, care asigură crearea unei singure instanțe globale per clasă, este delegată către containerul *Inversion of Control (IoC)* din framework-ul Spring.

Explicație și Justificare: Clasele marcate cu `@RestController` (`NotificareController`), `@Service` (`NotificariService`) și `@Component` (`NotificareDAO`) sunt înregistrate ca *Singleton Beans*. La pornirea sistemului, Spring inițializează o singură instanță a serviciului `NotificariService`. Chiar dacă aplicația generează sute de notificări concomitent la momentul în care mulți clienți folosesc aplicația JavaFX, toate cererile trec prin această unică instanță. Serviciile noastre sunt complet *stateless*, așadar această abordare este thread-safe și optimizează excelent consumul de memorie.

6 Concluzii

Arhitectura microserviciului **Notificări** confirmă utilitatea principiilor de separare a responsabilităților (*Separation of Concerns*). Prin implementarea modelului Hexagonal și definirea strictă a conceptelor DDD în stratul de domeniu, logica alertelor a fost complet decuplată de sistemul de gestiune a bazei de date. Utilizarea pattern-urilor Adaptor și Singleton asigură un cod ușor de testat, eficient din punct de vedere al resurselor și pregătit pentru cerințe viitoare.